CHARLES DARWIN UNIVERSITY

PRT565 Machine Learning, Artificial Intelligence and Algorithms

Assessment 4 · Group Report · 40% of the unit



PYRANTIS

Classifying early and late fire across the Northern Territory, and forecasting the 2026 season

---

Group	Group 85
Members	Harsh Rastogi (386401) Saira Zafar (407193) Tharushi Wimalachandra (387594)
Project lead	Harsh Rastogi
Campus	Darwin (Danala) Campus
Lecturer	Dr Asif Karim
Due	15 October 2026, 22:00 Sydney time

Assessment 3 recording: https://youtu.be/c_0yCsdZX2U
Live site: https://pyrantis.harshlabs.workers.dev
Code and data: github.com/harshrastogii/PYRANTIS (also in the submitted zip file)

Who did what

The work was divided by mutual agreement at a group meeting. Harsh Rastogi, as project lead, took half the report. Saira Zafar and Tharushi Wimalachandra took a quarter each. Each member wrote their own part, and all three reviewed the others' parts before submission. The word counts cover body text only. Headings, captions, tables, code, the cover, this page and the references are not included.

Member	Segments written	Words
Harsh Rastogi (386401)	Group information and project title; problem description and motivation; dataset description; results obtained and overall understanding, including the 2026 forecast, ethics and limitations (Sections 1–3 and 7)	3,121
Saira Zafar (407193)	Dataset analysis with charts and what each one showed; feature transformation, preprocessing, feature engineering and feature selection; k-means analysis (Section 4)	1,635
Tharushi Wimalachandra (387594)	Choice and description of the six algorithms and the baselines; description of key steps (split, class weights, early stopping, tuning); overview of model evaluation measures (Sections 5 and 6)	1,557
Total		**6,313**

Contents

1. Group information and project title 5
1.1 Group 85 5
1.2 Project title 5

2. Problem description and motivation 5
2.1 The problem 5
2.2 Why we chose it 6

3. Dataset description 7
3.1 Three sources on one grid 7
3.2 Rows, columns and classes 7
3.3 When the Territory burns 8
3.4 Fire and rain, year by year 9

4. Dataset analysis, feature selection, transformation, preprocessing and engineering 10
4.1 Where it burns 10
4.2 Fire history and the wet season together 11
4.3 The shape of the inputs and how we transformed them 12
4.4 How the inputs relate to each other 13
4.5 Fire regions found by k-means clustering 14
4.6 Preprocessing: from pixels to labels 15
4.7 Feature engineering and selection 16

5. Choice and description of the algorithms 17
5.1 Why these six 17
5.2 Decision tree 18
5.3 Random forest 18
5.4 Naive Bayes 18
5.5 Gradient boosting 18
5.6 Multilayer ANN 19
5.7 LSTM 19
5.8 Key steps 19

6. Overview of model evaluation measures 21
6.1 The confusion matrix 21
6.2 Accuracy 22
6.3 Precision 22
6.4 Recall 22
6.5 F1 score and macro averaging 22
6.6 Scoring the probabilities 22
6.7 Overfitting checks 23

7. Results and our overall understanding 23
7.1 How the six models compare 23
7.2 Precision and recall class by class 25
7.3 What class weighting did 26
7.4 Overfitting 26
7.5 Testing it sixty-six times 27
7.6 What the model relies on 28
7.7 The 2026 forecast 29
7.8 Our overall understanding 29
7.9 Limitations 30

References 31

1. Group information and project title

1.1 Group 85

Group 85 has three members, all enrolled at the Darwin (Danala) campus: Harsh Rastogi (386401), who led the project, Saira Zafar (407193) and Tharushi Wimalachandra (387594). The table on page 2 shows who wrote each part of this report and how many words each part holds.

1.2 Project title

PYRANTIS: classifying early and late fire across the Northern Territory, and forecasting the 2026 season.

This report is the detailed account of the project our group presented for Assessment 3. The data, models and results are the same ones presented in that recording (https://youtu.be/c_0yCsdZX2U). What the report adds is room: the analysis of the dataset that a fifteen-minute talk could not hold, a fuller description of each algorithm and the steps around it, the evaluation measures explained from first principles, and what we understood from the results once we had time to look at them closely. Every number below was produced by the scripts in the submitted zip file, from the data files packed with them. The interactive version of the results is at https://pyrantis.harshlabs.workers.dev and the code is at github.com/harshrastogii/PYRANTIS.

2. Problem description and motivation

2.1 The problem

In an average year about a third of the Northern Territory burns. Across 2000 to 2025 the share of the Territory's five-kilometre squares that carried a fire ranged from 15.9% in 2020 to 65.7% in 2011. With that much country burning every year, the useful question for a land manager is when a place will burn, more than whether it will.

Timing decides what a fire does. Up to the end of July the grass still holds some moisture, nights are cool and the wind is light. Fires in that window are patchy, slow and often lit on purpose. After July the grass has cured, humidity drops and the south-easterlies pick up, so the same ignition burns hotter, spreads further and kills canopy, fences and stock (Russell-

Smith et al., 2007). Savanna burning across northern Australia is built on this difference. Ranger groups and pastoralists burn early to break up the fuel before the late season arrives, and savanna carbon projects are paid for shifting fire from late to early (Russell-Smith et al., 2013). North Australian Fire Information (NAFI) publishes a separate late-season fire layer on the same 31 July boundary.

We set the task up as supervised multi-class classification, the first type of learning covered in Week 1. For every square about five kilometres across, and for every year, the model predicts one of three labels:

- unburnt: no fire was mapped in the square that year;
- early: the square burnt, and most of the burning was on or before 31 July;
- late: the square burnt, and most of the burning was after 31 July.

Two rules shape the whole project. The first is that the model may use only information that existed by 30 April of the year it predicts, which is the end of the wet season. A prediction made under that rule exists before the fire season starts, so a ranger could act on it. A model allowed to see August weather would describe the fire season rather than forecast it. The second rule is that every score is measured on years the model never saw during training, because the model is only useful for years that have not happened yet.

2.2 Why we chose it

We chose this problem for four reasons. The first is that it is local. NAFI is run from Charles Darwin University, and the fire seasons the data records are the ones that fill Darwin with smoke every September. The second is that all three data sources are public and well documented, so anyone with the zip file can rebuild every number in this report. The third is that the problem has the kind of class imbalance the unit spends time on. Late fire is the class that matters most, and it is only 16.9% of the modelling rows, so accuracy alone would reward a model that ignores it. That made precision, recall and F1 the measures that decide the project, which gave Week 5 real work to do. The fourth is that the forecast can be checked. NAFI will publish its 2026 fire mapping in 2027, and our 2026 prediction, made from the 2025–26 wet season, will either hold up or it will not.

3. Dataset description

3.1 Three sources on one grid

The dataset is built from three public archives, joined on a single grid of 0.05-degree cells. A cell is about 5.1 km east to west and 5.5 km north to south, and 46,445 of them fall inside the Territory with enough mapped ground to label.

Table 1. The three data sources and what each contributes.

Source	Provider	What it gives	Resolution	Used
NAFI fire scars	Charles Darwin University	Month each pixel was mapped as burnt	250 m, yearly	2000–2025
SILO Data Drill	Queensland Government	Daily rain, max temperature, VPD, radiation; elevation	497 points, 0.5° apart, daily	1999 to April 2026
MODIS MOD13Q1	NASA, via Google Earth Engine	Vegetation greenness (NDVI)	250 m, every 16 days	2000 to April 2026

NAFI's burnt-area rasters are the source of the labels. Each 250-metre pixel carries the month in which it was mapped as burnt, from 1 to 12, or zero for no fire, which is what makes an early or late label possible at all (North Australian Fire Information, 2026). Twenty-six annual rasters cover 2000 to 2025.

SILO's Data Drill interpolates Bureau of Meteorology station records onto a national grid (Jeffrey et al., 2001; Queensland Government, 2026). We requested daily rainfall, maximum temperature, vapour pressure deficit and solar radiation at 497 points spaced half a degree apart across the Territory, and interpolated them linearly onto the 0.05-degree cells. Weather changes slowly over 50 km in the Territory, and 497 requests is a fair load to put on a free public service, where 46,000 would not be. SILO also reports the elevation of each point, which became one of the inputs.

The MODIS MOD13Q1 product gives vegetation greenness (NDVI) at 250 m every sixteen days (Didan, 2021). We extracted it through Google Earth Engine (Gorelick et al., 2017), averaged straight onto our grid. Greenness measures the grass itself, where rainfall only measures what grew it.

3.2 Rows, columns and classes

Labelling produced 1,207,570 cell-years across 2000 to 2025. The modelling table is smaller, at 966,819 rows covering 2005 to 2025. The first five years are held back so that every row has at least five years of fire history behind it, and cells without a complete ring of eight neighbours are dropped because the neighbour features cannot be computed for them. The

table has 57 columns: identifiers (cell, row, column, year), the label and the target-year burnt fraction, which are never given to a model, and the 50 input features described in Section 4. It has no missing values.

The years are split by time. Training uses 2005 to 2019 (690,585 rows), tuning and early stopping use 2020 to 2022 (138,117 rows), and the final test uses 2023 to 2025 (138,117 rows), scored once after every decision had been made.

Table 2. Class shares in each part of the modelling table.

Part	Years	Rows	Unburnt	Early	Late
Training	2005–2019	690,585	69.6%	13.5%	16.9%
Validation	2020–2022	138,117	77.1%	14.5%	8.4%
Test	2023–2025	138,117	57.8%	16.6%	25.5%
All modelling rows	2005–2025	966,819	69.0%	14.1%	16.9%
All labels	2000–2025	1,207,570	67.5%	14.0%	18.5%

Table 2 shows two things the rest of the report depends on. Unburnt dominates every period, so a model that always says "no fire" is right 58% to 77% of the time while learning nothing. The class mix also moves between periods. Late fire was 8.4% of the validation years and 25.5% of the test years, three times as common. The years we tuned on were quiet, and the years we tested on included 2023, when 37.1% of the Territory burnt late. Section 7 comes back to what that shift did to the results.

3.3 When the Territory burns

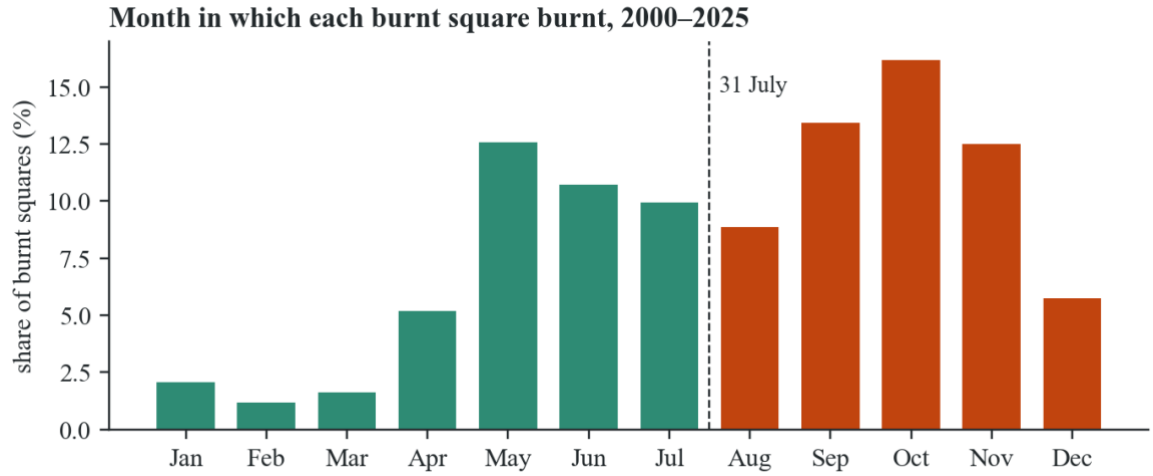


Figure 1. The month in which each burnt square burnt, across all 26 years.

Figure 1 is the reason the label has two fire classes. Burning does not spread evenly through the year. It has a first peak in May (12.6% of all burnt squares), dips to 8.9% in August, and then climbs to its main peak in October (16.2%). August to November holds 51.0% of the

burning, and May to July holds 33.2%. A single "burnt" class would mix these two seasons, which differ in how hot the fires are, how much damage they do and whether people lit them on purpose. The chart also shows that 10.1% of burning happens between January and April. Those squares count as early under the 31 July rule, which is correct for the management question even though they are wet-season fires.

3.4 Fire and rain, year by year

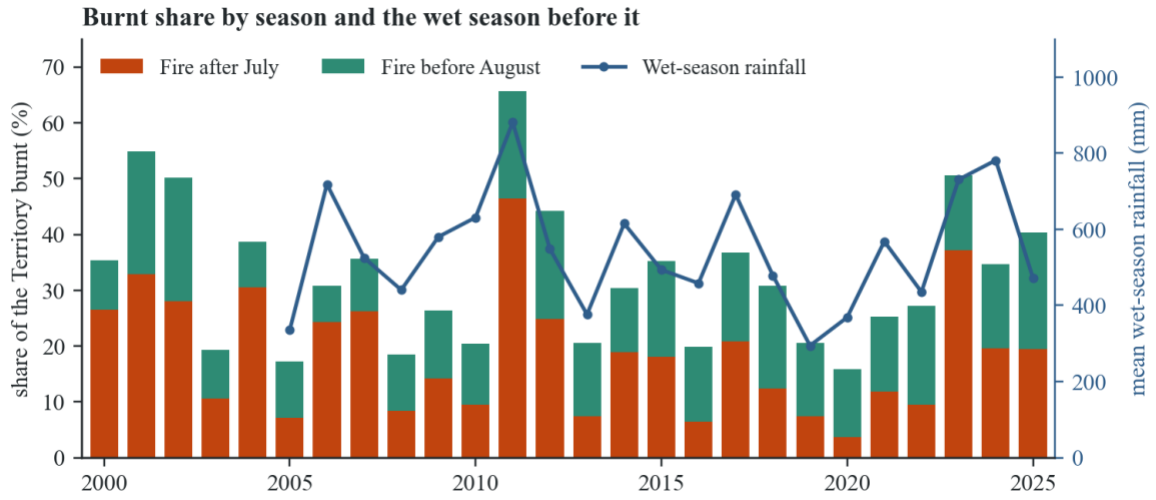


Figure 2. Share of the Territory burnt each year, split by season, against the Territory-wide mean of wet-season rainfall (November to April).

Figure 2 combines three variables: year, season and rainfall. The late share swings far more than the early share. In the driest years late fire covered 3.7% of the Territory (2020), and in the wettest it covered 46.5% (2011). Early fire stayed between 6% and 22% throughout. Across 2005 to 2025 the correlation between wet-season rainfall and the late share in the same year is 0.79, while for the early share it is 0.09. Our reading is that early fire is mostly lit by people, every year, whatever the rain did. Late fire depends on how much grass the wet season grew. The big late years, 2011 after the 2010–11 La Niña and 2023 after a wet 2022–23, each follow one of the wettest wet seasons in the record. This chart told us that rainfall features would matter before any model had been trained. Section 4 takes the analysis further, from where fire happens to how the inputs relate to each other.

4. Dataset analysis, feature selection, transformation, preprocessing and engineering

Section 3 described the target. This section analyses the inputs, following the order of the Week 6 practical: single features, then pairs, then several at once. Below each chart is what we learned from it that the raw table did not show.

4.1 Where it burns

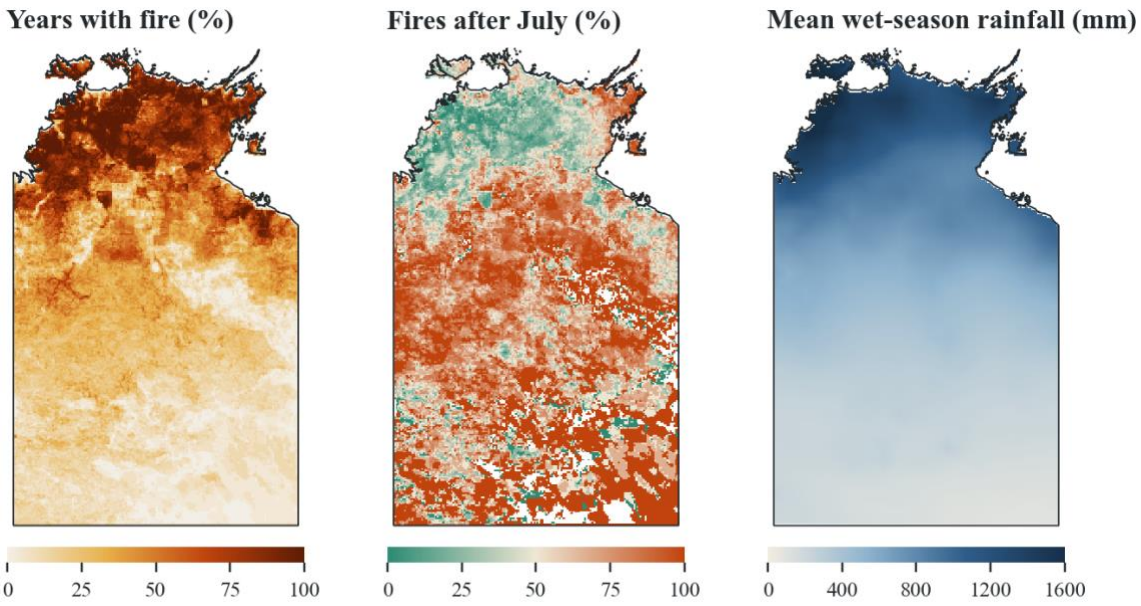


Figure 3. Left: share of years in which each square burnt. Centre: share of a square's fires that came after July (white squares never burnt). Right: mean wet-season rainfall.

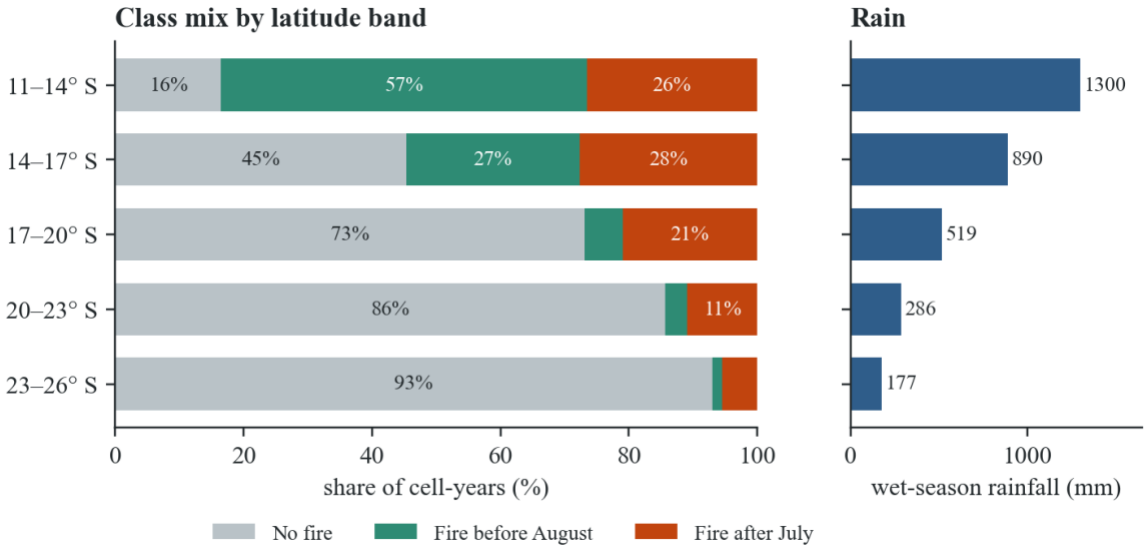


Figure 4. Class mix by latitude band, beside the mean wet-season rainfall of each band.

Figure 3 shows that the Territory has several fire regimes. North of 14° S a square burnt in 82.9% of years on average, and only 36.2% of those fires came after July. South of 20° S a square burnt in 12.4% of years, but 73.7% of those fires were late. Rainfall follows the same north to south gradient as fire frequency. Figure 4 puts numbers on it. The wettest band (1,300 mm) is unburnt only 16.5% of the time, and most of its fire is early. The driest band (177 mm) is unburnt 93.0% of the time, and the little fire it has is mostly late. We took two things from this. Latitude, rainfall and greenness overlap heavily, since all three track the same gradient. And a model has to be able to tell the wet north from the arid south, which is why longitude, latitude and elevation were kept as features.

4.2 Fire history and the wet season together

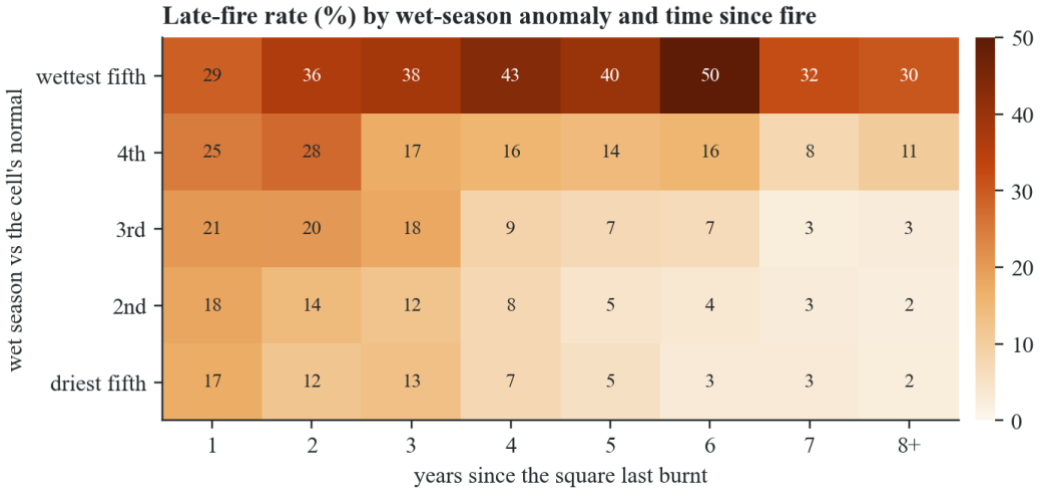


Figure 5. Percentage of cell-years with a late fire, by how wet the season was compared with the cell's own normal (rows, in fifths) and by years since the square last burnt (columns).

Figure 5 was the most useful chart we made, because it shows two features interacting. The rows split the wet-season rainfall anomaly into fifths. The anomaly compares each cell with its own 2000–2019 average, so a wet year in the desert and a wet year in Arnhem Land both count as wet. The columns give the years since the square last burnt. In the driest fifth, the late-fire rate falls from 17.4% for a square that burnt last year to 2.4% for one unburnt for eight years or more. In the wettest fifth it stays between 29% and 50% at every interval, peaking six years after the last fire.

Neither feature explains this alone. Long-unburnt country in a normal year is mostly the arid south, where there is too little grass to carry fire. After a wet season the same country grows grass it has not had for years, and then it burns late. Tree-based models learn this kind of interaction without being told about it (Section 5). The chart also explains a pattern that looks

wrong at first. Squares that burnt last year are the most likely to burn again (53.9% do, 32.2% early), because most of them are northern squares burnt in planned programmes every year.

4.3 The shape of the inputs and how we transformed them

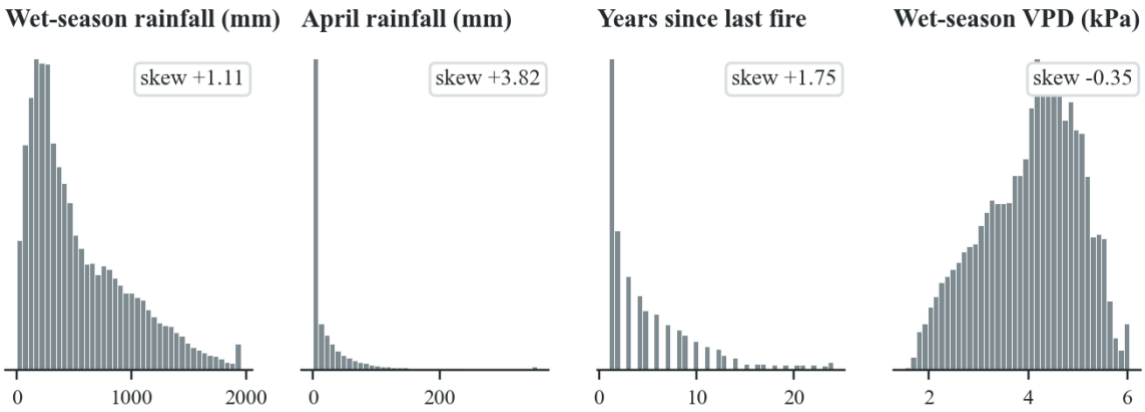


Figure 6. Distributions of four inputs with their skewness, following the Week 6 skewness check (values above the 99.5th percentile are clipped for display only).

Week 6 showed how a skewed feature can be reshaped toward a normal distribution, so we checked every input. Three are clearly right-skewed: April rainfall (skewness 3.82), years since the last fire (1.75) and wet-season rainfall (1.11). Wet-season vapour pressure deficit is close to symmetric (−0.35). A log transform would bring April rainfall down to 0.44 but over-correct wet-season rainfall to −0.66.

We did not transform the features for the tree models. A tree splits on thresholds, and a monotonic transform such as a logarithm moves the thresholds without changing which rows fall on each side, so trees, forests and boosting give the same result either way. The neural networks learn by gradient descent (Week 2), which works poorly when one input runs to 1,500 and another to 1. For them every feature was standardised with StandardScaler, as in Week 3, fitted on the training years only so the later years could not influence the scaling. Naive Bayes assumes each feature is normal within each class, which the skewed features break.

The Week 3 encoding step applied to the target only. The three text labels were mapped to the integers 0, 1 and 2. None of the inputs is categorical, because last year's state is stored as three separate 0/1 indicator columns, which is one-hot encoding in practice.

4.4 How the inputs relate to each other

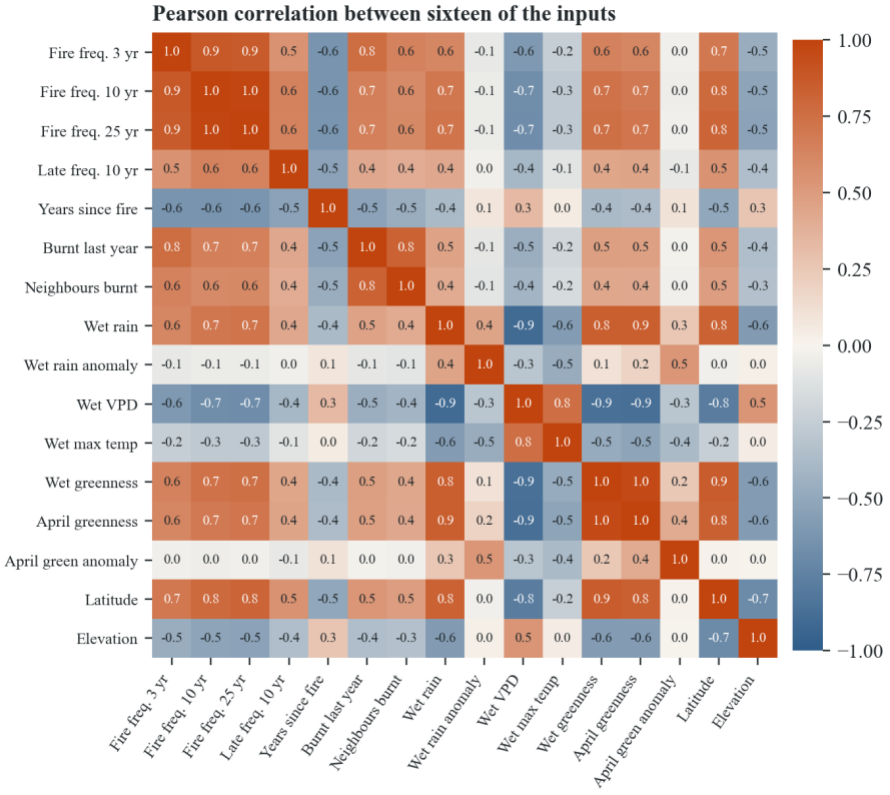


Figure 7. Pearson correlation between sixteen of the fifty inputs.

Figure 7 shows heavy redundancy. Fire frequency over ten and over twenty-five years correlate at 0.98. Mean wet-season greenness and April greenness correlate at 0.96. Wet-season rainfall and vapour pressure deficit correlate at −0.90, because wet air is humid air. Latitude correlates with wet-season greenness at 0.85, the gradient of Figure 3 again. The rainfall anomaly correlates with raw rainfall at only 0.45, so it tells the model something the raw total does not, which is how unusual the year is for that place.

Correlated features are a problem for Naive Bayes, which assumes the features are independent within each class, and less of one for trees, which use whichever of two near-copies splits better. The redundancy also explains why many features showed low importance later. When two features carry the same information, shuffling one costs the model little, because it can still read the other.

4.5 Fire regions found by k-means clustering

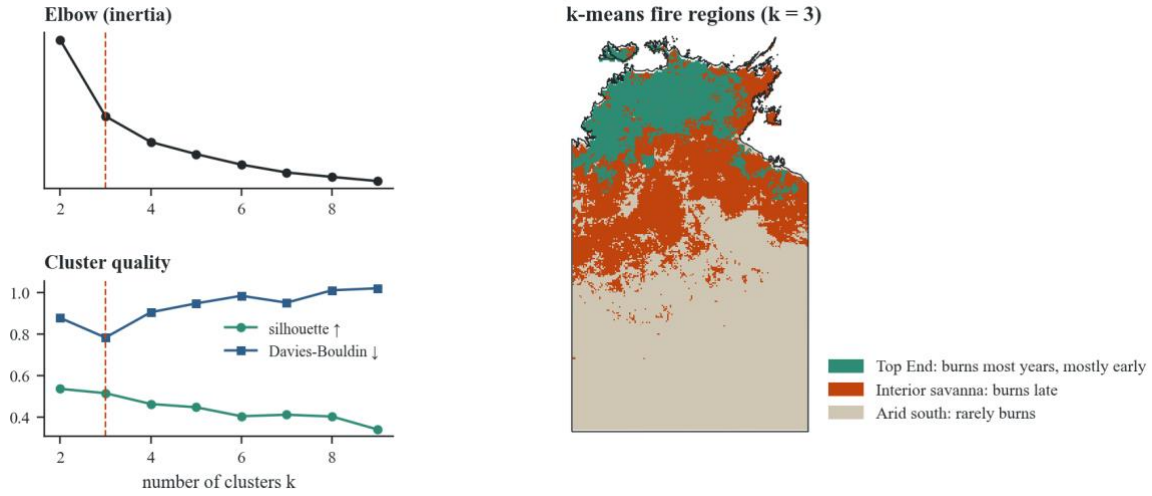


Figure 8. k-means clustering of the 46,039 modelled squares on early-fire frequency, late-fire frequency and mean wet-season rainfall, with the elbow, silhouette and Davies-Bouldin checks from the Week 11 practical.

To check whether the regimes in Figure 3 are real groups or a smooth gradient, we clustered the squares with k-means (Lloyd, 1982), following the Week 11 practical. Each square was described by three standardised numbers: how often it burnt early, how often it burnt late, and its mean wet-season rainfall. We tried k = 2 to 9. The elbow in inertia falls at k = 3. The Davies-Bouldin index (Davies & Bouldin, 1979), where lower is better, is lowest at k = 3 (0.783). The silhouette score (Rousseeuw, 1987), where higher is better, is 0.514 at k = 3, close to its best of 0.535 at k = 2. All three checks point to three groups.

The Top End group (6,486 squares, 1,194 mm of wet-season rain) burns in 84.9% of years, mostly early (62.0% of years early against 22.9% late). The interior savanna group (13,230 squares, 737 mm) burns in 46.2% of years, three times as often late as early. The arid south group (26,323 squares, 287 mm) burns in 13.0% of years. The boundaries are ragged, but the Territory clearly splits into regions with different fire behaviour, so one rule for the whole Territory would not fit. The clusters were used to understand the data only. They were not given to the models, because they were fitted on all 26 years, test years included.

4.6 Preprocessing: from pixels to labels

scripts/build_grid.py lines 102–110, lines 125–128

```
102 transform, ny, nx, inside = grid_meta
103 burnt = (arr != UNBURNT_VALUE) & inside
104 early = burnt & (arr < LATE_SEASON_FIRST_MONTH)
105 late = burnt & (arr >= LATE_SEASON_FIRST_MONTH)
106
107 blk = lambda a: a.reshape(ny, BLOCK, nx, BLOCK).sum(axis=(1, 3))  # noqa: E731
108 n_early, n_late = blk(early), blk(late)
109 n_valid = blk(inside)
110 n_burnt = n_early + n_late
…
125 label = np.full((ny, nx), CLASSES[0], dtype=object)
126 is_burnt = frac_burnt >= MIN_BURNT_FRACTION
127 label[is_burnt & (n_early >= n_late)] = "early"
128 label[is_burnt & (n_late > n_early)] = "late"
```

Figure 9. The labelling rule in build_grid.py. Pixels are pooled into 20 × 20 blocks, and burnt pixels are counted separately for each side of 31 July.

Each 0.05-degree cell is a block of 20 × 20 NAFI pixels, and Figure 9 shows the rule that turns a block into a label. A pixel mapped in months 1 to 7 counts as early, and one mapped in months 8 to 12 counts as late. A cell is labelled burnt only when at least 5% of its readable pixels burnt, because one or two stray pixels in a block of 400 are within the noise of the mapping. A burnt cell takes whichever season holds more of its burnt pixels. Cells where less than half the block lay inside the Territory and NAFI's mapped area were dropped, or mostly-sea coastal cells would be labelled unburnt on the evidence of a handful of pixels.

No metric can catch a wrong label, so we checked the labels against the weather. The pipeline puts 2011 at 65.7% burnt, the heaviest year in the record, and 2020 at 15.9%, the lightest. The 2011 season followed the 2010–11 La Niña, the wettest wet season in our record (881 mm on average), and 2020 followed the two driest (294 mm and 369 mm). A labelling error would be unlikely to line up with the rainfall record this well.

Most of the cleaning steps from Weeks 3 and 6 were not needed. The table has no missing values, because rows without five years of history or a full ring of neighbours were removed rather than filled, so there was nothing to impute. Outliers were kept. A 1,400 mm wet season in the desert is extreme, but it happened, and it is exactly the year a late fire follows. Removing it would delete the signal we were looking for.

4.7 Feature engineering and selection

scripts/build_features.py lines 70–80, lines 100–109

```
70 def features_at(t, burnt, early, late, frac_burnt, frac_early, frac_late):
71     """Every feature for target year index t, from years strictly before it.
72
73     Pulled out of the yearly loop so the forecast path computes the same numbers the
74     training path does. A second implementation would drift, and the drift would be
75     silent: the forecast has no label to check itself against.
76     """
77     # Everything below slices [:t] -- strictly years before the target year.
78     past_burnt = burnt[:t]
79     past_late = late[:t]
…
100     f[f"never_{name}"] = (~ever).astype("f4")
101
102 # Last year's state, the single strongest fire-history signal: a place that
103 # burnt last year has little fuel left to carry a fire this year.
104 f["prev_burnt"] = burnt[t - 1]
105 f["prev_early"] = early[t - 1]
106 f["prev_late"] = late[t - 1]
107 f["prev_frac_burnt"] = frac_burnt[t - 1]
108 f["prev_frac_early"] = frac_early[t - 1]
109 f["prev_frac_late"] = frac_late[t - 1]
```

Figure 10. Part of the shared feature function in build_features.py. Every history feature is sliced from years strictly before the target year t.

Week 6 described feature engineering as combining existing features, or bringing in outside ones, to help the model. Almost every input here is engineered, because the raw data is a stack of yearly rasters that no classifier can read. Figure 10 shows how the fire-history features are built. A single function computes them from the years before the target year only (the [:t] slice), and both the training path and the forecast path call it. Two separate copies could drift apart, and the forecast has no label that would reveal the drift.

Table 3. The 50 input features, in four groups. All are known by 30 April of the year predicted.

Group	Count	What it holds
Fire history	26	Fire frequency and late-fire frequency over 3, 5, 10, 15, 20 and 25 years; years since the last fire and last late fire, with never-burnt flags; last year's state and burnt fractions; burnt two years ago; what the eight neighbouring cells did last year
Weather	14	Wet-season (November to April) rainfall, maximum temperature, vapour pressure deficit and radiation; April rainfall and VPD; days since meaningful and heavy rain; the previous wet and dry seasons; rainfall anomaly and ratio against the cell's 2000–2019 normal
Greenness	7	Mean, highest and lowest wet-season NDVI; April NDVI; green-up speed; April and peak anomalies against the cell's 2005–2019 normal
Place	3	Longitude, latitude and elevation

Table 3 lists the four groups. The fire-history group turns the label sequence into information about fuel. The weather group summarises the wet season (November to April) and the dry season before it, with anomaly and ratio features against the cell's own 2000–2019 normal.

The greenness group measures the grass directly, with anomalies against each cell's 2005–2019 average, the training years only. Two kinds of input were left out on purpose. Anything measured in the target year, such as that year's burnt fraction, would hand the model the answer. Each cell's long-run average greenness was dropped too, because a per-cell constant acts as an identifier for the cell rather than evidence about the year.

Features were selected by experiment, following the Week 7 advice to combine domain knowledge with a measured check of performance. Each candidate block was added to a fixed benchmark and kept only if it raised macro F1 on the validation years. Twenty-three experiments were run and recorded, failures included. District-scale neighbourhood features made the model worse and were dropped. Longer fire memory (15, 20 and 25 years), elevation and greenness were kept. On the test years, the best model using fire history alone reached a macro F1 of 0.639, adding weather lifted the best score to 0.674, and the final 50 features reached 0.684.

The tests in tests/test_no_leakage.py check the pipeline for the leakage Week 3 warned about. One rebuilds sampled features from the raw labels and compares them. The other shuffles one year's labels, rebuilds every feature and confirms that no feature for that year changes, which a feature secretly reading the target year would fail.

Written by Tharushi Wimalachandra (387594) Sections 5 and 6

5. Choice and description of the algorithms

5.1 Why these six

The Assessment 3 brief asked for a decision tree, a random forest, Naive Bayes and a multilayer artificial neural network. We trained those four and added two. Gradient boosting was added because tree ensembles are usually the strongest choice for tabular data with interacting features, and Figure 5 had shown that ours interact. The LSTM was added because each square's fire history is a sequence, the kind of data Week 9 used recurrent networks for. All six were trained on the same 690,585 rows and scored on the same test years, using scikit-learn (Pedregosa et al., 2011) and Keras (Chollet, 2015).

Two baselines need no learning. "Always unburnt" predicts the majority class. "Same as last year" predicts whatever the square did the year before, roughly what an experienced fire officer would assume. A model that cannot beat the second has learned nothing useful.

5.2 Decision tree

A decision tree splits the data with a sequence of yes/no questions about single features, such as "did more than 30% of the neighbours burn last year?", choosing at each node the question that leaves the two halves purest. Week 4 measured purity with entropy and information gain, and the Gini index in scikit-learn's CART works the same way (Breiman et al., 1984; Quinlan, 1986). Every prediction can be traced to a short chain of thresholds, which a fire manager can read. Its weakness, as Week 4 warned, is overfitting. Our hand-set tree had a maximum depth of 12 and at least 50 rows per leaf. Tuning chose 500-row leaves and a depth of 20. Larger leaves act as pruning and stop the tree memorising individual years.

5.3 Random forest

A random forest trains many trees, each on a bootstrap sample of the rows, with each split choosing from a random subset of the features, and takes a majority vote (Breiman, 2001). Week 4 called this the wisdom of crowds: trees that make different mistakes cancel each other out. Our forest used 200 trees with at least five rows per leaf. Tuning moved it to 400 trees, a depth of 30 and the square root of the feature count at each split.

5.4 Naive Bayes

Gaussian Naive Bayes applies Bayes' theorem, assuming every feature is independent of the others within a class and normally distributed. It estimates a mean and variance per feature per class and picks the class with the highest posterior probability (Week 5). It trains in under a second and sets a useful floor. Section 4 showed its assumptions fail here. Several features correlate above 0.9 (Figure 7), so it counts the same evidence more than once, and several are skewed (Figure 6). We gave it equal class priors, its version of class weighting.

5.5 Gradient boosting

Gradient boosting builds shallow trees in sequence. Each new tree is fitted to the errors the ensemble has made so far, and its contribution is shrunk by a learning rate, so the model improves in small corrections (Friedman, 2001). It is the gradient descent of Week 2, applied to trees instead of weights. We used HistGradientBoostingClassifier, which sorts each feature into at most 255 bins so that 690,585 rows train in about a minute and a half (Ke et al., 2017). The hand-set model used 500 trees, a learning rate of 0.07, 63 leaves per tree, 40 rows per leaf and L2 regularisation of 1.0. Regularisation (Week 3) penalises large leaf values so the model cannot chase noise.

5.6 Multilayer ANN

The multilayer ANN is a feed-forward network of the kind shown in Week 8: 50 standardised inputs, two hidden layers of 128 and 64 ReLU neurons, and a softmax output giving a probability for each class. It is trained by backpropagation, which passes the output error back through the layers to adjust every weight (Rumelhart et al., 1986), with the Adam optimiser and cross-entropy loss (Kingma & Ba, 2015). Dropout of 20% after each hidden layer switches off a random fifth of the neurons at each step, as Week 9 did to stop an MLP memorising MNIST (Srivastava et al., 2014).

5.7 LSTM

A long short-term memory network reads a sequence one step at a time and keeps a memory cell whose gates decide what to keep and what to forget (Hochreiter & Schmidhuber, 1997). Week 9 fed it a sliding window over a time series. Ours reads the last ten years of each square's fire state, six numbers per year (burnt, early and late fractions, and the three class indicators), stopping one year before the target. A sequence alone carries no weather, which would be unfair to the LSTM, so a second branch reads the same 50 features as the other models and the two are joined before the output (Figure 12).

5.8 Key steps

scripts/train_models.py lines 201–204, lines 233–245

```
y_all = feat["label"].map({c: i for i, c in enumerate(CLASSES)}).to_numpy()
tr = feat["year"].isin(TRAIN_YEARS).to_numpy()
va = feat["year"].isin(VAL_YEARS).to_numpy()
te = feat["year"].isin(TEST_YEARS).to_numpy()
…
specs = [
    ("decision tree", lambda w: DecisionTreeClassifier(
        max_depth=12, min_samples_leaf=50, class_weight=w, random_state=SEED)),
    ("random forest", lambda w: RandomForestClassifier(
        n_estimators=200, min_samples_leaf=5, n_jobs=-1,
        class_weight=("balanced_subsample" if w else None), random_state=SEED)),
    ("naive bayes", lambda w: GaussianNB(
        priors=(np.full(len(CLASSES), 1 / len(CLASSES)) if w else None))),
    ("gradient boosting", lambda w: HistGradientBoostingClassifier(
        max_iter=500, learning_rate=0.07, max_leaf_nodes=63, l2_regularization=1.0,
        min_samples_leaf=40, early_stopping=False,
        class_weight=("balanced" if w else None), random_state=SEED)),
]
```

Figure 11. The temporal split and the four classical models with their class weights, from train_models.py.

Figure 11 shows the two steps that matter most. The first is the split by year. Week 10 taught k-fold cross-validation, which shuffles rows into folds and assumes they are independent. Ours are not. A square in 2012 looks like its neighbour in 2012 and like itself in 2013, so a random split puts near-copies of the test rows into training (Roberts et al., 2017). We

measured the effect. Repeated stratified 5-fold cross-validation on 240,000 training rows scored gradient boosting at 0.772, while the same model scored 0.677 on later years.

The second step is class weighting. Every model was trained twice, once as it is and once with each class weighted in inverse proportion to its frequency (He & Garcia, 2009). Unweighted, a model gains accuracy by rarely predicting late fire. Weighted, a missed late fire costs about four times what a wrong unburnt costs. The weighted versions are the ones reported.

scripts/train_models.py lines 286–294, lines 307–316

```
286 def ann():
287     return keras.Sequential([
288         keras.layers.Input((len(features),)),
289         keras.layers.Dense(128, activation="relu"),
290         keras.layers.Dropout(0.2),
291         keras.layers.Dense(64, activation="relu"),
292         keras.layers.Dropout(0.2),
293         keras.layers.Dense(len(CLASSES), activation="softmax"),
294     ])
…
307 def lstm():
308     seq_in = keras.layers.Input((SEQ_LEN, seq.shape[2]), name="history")
309     sta_in = keras.layers.Input((len(features),), name="static")
310     h = keras.layers.LSTM(64)(seq_in)
311     h = keras.layers.Dropout(0.2)(h)
312     g = keras.layers.Dense(32, activation="relu")(sta_in)
313     z = keras.layers.Concatenate()([h, g])
314     z = keras.layers.Dense(32, activation="relu")(z)
315     out = keras.layers.Dense(len(CLASSES), activation="softmax")(z)
316     return keras.Model([seq_in, sta_in], out)
```

Figure 12. The multilayer ANN and the two-branch LSTM, from train_models.py.

Both networks used early stopping. After each epoch the model scored macro F1 on the validation years, kept its best weights and stopped after six epochs without improvement. We stopped on macro F1 rather than loss because the validation years were quiet (8.4% late), and their loss rose almost at once even while the model was still improving.

scripts/robustness.py lines 195–198, lines 219–225

```
195 itr = RNG.choice(np.flatnonzero(tr), 200000, replace=False)
196 iva = RNG.choice(np.flatnonzero(va), 100000, replace=False)
197 Xs = np.vstack([X[itr], X[iva]]); ys = np.concatenate([y[itr], y[iva]])
198 split = PredefinedSplit(np.r_[np.full(len(itr), -1), np.zeros(len(iva), int)])
…
219 base = clone(CLASSICAL[name])
220 if kind == "grid":
221     s = GridSearchCV(base, space, scoring=F1, cv=split, n_jobs=1, refit=False)
222 else:
223     s = RandomizedSearchCV(base, space, n_iter=n_iter, scoring=F1, cv=split,
224                            random_state=T.SEED, n_jobs=(1 if name == "random forest" else -1),
225                            refit=False)
```

Figure 13. Tuning with a single forward-in-time fold, from robustness.py.

Each hand-set model was then tuned with the Week 10 tools (Figure 13): RandomizedSearchCV for the three tree models (Bergstra & Bengio, 2012), GridSearchCV for Naive Bayes, and a small grid over size, dropout and learning rate for the networks.

Instead of k folds, a PredefinedSplit gave one fold that trained on 200,000 sampled training rows and scored on 100,000 validation rows, so every candidate was judged on later years and the test years played no part.

Table 4. Systematic tuning. Settings were chosen on the validation years only, and the test years were used once.

Model	Search	Tried	Best settings	Val F1	Test F1 tuned	Test F1 hand-set
Gradient boosting	Randomised	14	rows per leaf 100, leaves 63, trees 800, learning rate 0.05, L2 3.0	0.679	0.684	0.684
LSTM	Grid	4	units 64, dropout 0.4, learning rate 0.001	0.673	0.670	0.670
Multilayer ANN	Grid	7	units 256/128, dropout 0.2, learning rate 0.001	0.678	0.674	0.667
Random forest	Randomised	10	trees 400, rows per leaf 5, features per split sqrt, depth 30	0.685	0.662	0.660
Decision tree	Randomised	25	rows per leaf 500, depth 20, criterion gini	0.643	0.632	0.604
Naive Bayes	Grid	11	variance smoothing 1e-12	0.639	0.603	0.602

Tuning changed little. Gradient boosting scored 0.684 on the test years with both settings. The decision tree gained most, from 0.604 to 0.632. No other model moved by more than 0.007. Section 7 reports the hand-set models, which match the Assessment 3 presentation.

6. Overview of model evaluation measures

6.1 The confusion matrix

Every measure we report comes from the confusion matrix (Week 5), whose rows are what happened and whose columns are what the model predicted. For one class, say late fire, a true positive (TP) is a late fire called late. A false positive (FP) is a square called late that did not burn late, a false alarm. A false negative (FN) is a late fire called something else, a missed fire. A true negative (TN) is everything else. With three classes, each class takes its turn as the positive one.

scripts/train_models.py lines 111–123

```python
def score(name: str, y_true, y_pred, seconds: float) -> dict:
    lbl = list(range(len(CLASSES)))
    return {
        "model": name,
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision_macro": float(precision_score(y_true, y_pred, average="macro", zero_division=0)),
        "recall_macro": float(recall_score(y_true, y_pred, average="macro", zero_division=0)),
        "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
        "f1_per_class": dict(zip(CLASSES, [
            float(f) for f in f1_score(y_true, y_pred, average=None, labels=lbl, zero_division=0)])),
        "confusion": confusion_matrix(y_true, y_pred, labels=lbl).tolist(),
        "train_seconds": round(seconds, 1),
    }
```

Figure 14. The scoring function used for every model, from train_models.py.

6.2 Accuracy

Accuracy is the share of all predictions that are correct, (TP + TN) / (TP + TN + FP + FN). Week 5 noted that it is fair only when classes are balanced. Ours are not. On the test years the always-unburnt baseline scores 0.578 accuracy without detecting a single fire, so no decision in the project was made on accuracy.

6.3 Precision

Precision asks "of the squares the model called late, what share really burnt late?". It is TP / (TP + FP). High precision means few false alarms, which matters when a false alarm is expensive, such as sending a crew to burn a block that would never have carried a late fire.

6.4 Recall

Recall asks "of the squares that really burnt late, what share did the model catch?". It is TP / (TP + FN). High recall means few missed fires. For late fire it matters most to a land manager, because a missed late fire is country nobody prepared.

6.5 F1 score and macro averaging

Precision and recall pull against each other. A model that calls every square late has perfect late recall and poor precision. F1 is their harmonic mean, 2 × precision × recall / (precision + recall), and is high only when both are. With three classes we macro-average: precision, recall and F1 are computed per class and averaged, so the rare late class weighs as much as unburnt (Sokolova & Lapalme, 2009). Macro F1 was our main measure, and tuning, early stopping and the feature experiments all used it.

6.6 Scoring the probabilities

The measures above judge only the final answer. Each model also gives a probability for each class, and ROC-AUC judges those probabilities at every cut-off. It is the chance that the model ranks a square from a class above one from outside it, so guessing scores 0.5. PR-AUC

(average precision) is stricter for a rare class, since guessing scores the class's share, 0.255 for late fire on the test years (Saito & Rehmsmeier, 2015). Both were computed one class against the rest and averaged.

6.7 Overfitting checks

Week 3 defined overfitting as learning noise in the training data that does not carry over to new data. We measured it by scoring every model on its training years as well as on the validation and test years. A large gap between training and test F1 is the sign.

Written by Harsh Rastogi (386401) Section 7

7. Results and our overall understanding

7.1 How the six models compare

Table 5. Held-out results on 2023–2025. Precision, recall and F1 are macro-averaged. All six models are the class-weighted versions.

Model	Accuracy	Precision	Recall	F1	F1 late	ROC-AUC	PR-AUC
Gradient boosting	**0.725**	**0.679**	**0.690**	**0.684**	**0.553**	**0.862**	**0.732**
LSTM	0.692	0.679	0.687	0.673	0.561	0.854	0.724
Multilayer ANN	0.678	0.670	0.695	0.667	0.556	0.856	0.713
Random forest	0.727	0.688	0.651	0.660	0.468	0.859	0.730
Decision tree	0.628	0.598	0.632	0.604	0.495	0.793	0.633
Naive Bayes	0.644	0.592	0.616	0.602	0.398	0.797	0.637
Baseline: same as last year	0.567	0.511	0.504	0.506	0.241	–	–
Baseline: always unburnt	0.578	0.193	0.333	0.244	0.000	–	–

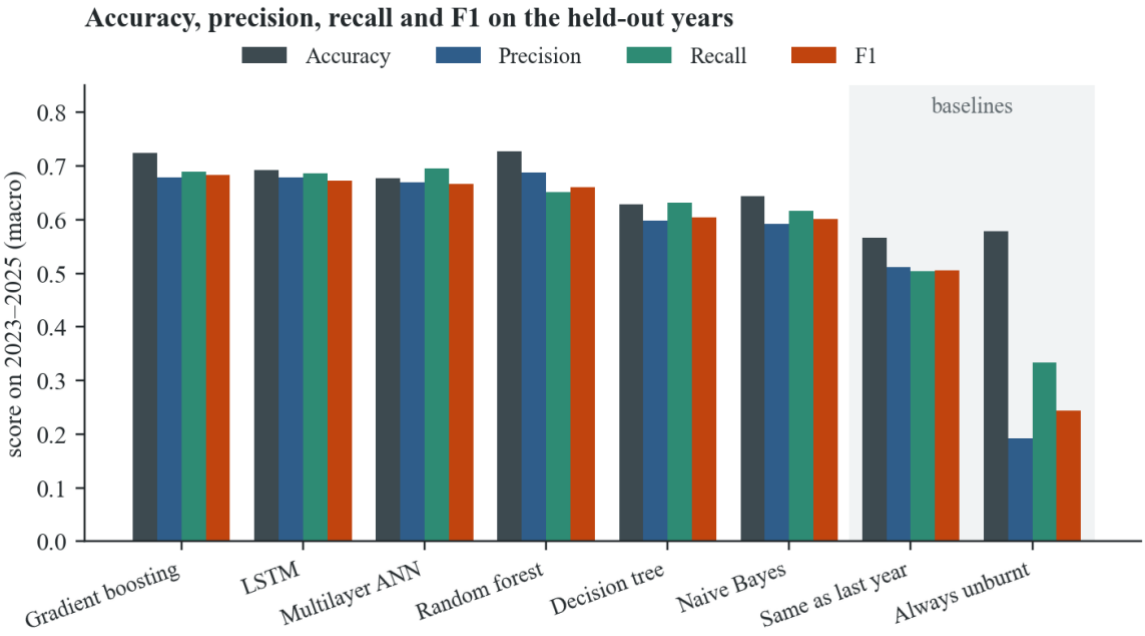


Figure 15. The four measures for every model and both baselines.

Gradient boosting scored best, with a macro F1 of 0.684, accuracy of 0.725, precision of 0.679 and recall of 0.690. The LSTM was second at 0.673 and the multilayer ANN third at 0.667. The gap between the learned models is smaller than the gap between the learned models and the baselines. "Same as last year" scores 0.506, so the best model adds 0.178 of macro F1 over what a fire officer would assume without one.

Accuracy and F1 rank the models differently, which is why we relied on F1. The random forest has the highest accuracy of all (0.727) but only the fourth-best F1 (0.660). It gets its accuracy by calling unburnt squares well (recall 0.894) and missing late fires (recall 0.384). The multilayer ANN does the opposite. It has the lowest accuracy of the top four (0.678) but the highest recall (0.695), because it calls late fire readily. If we had ranked by accuracy we would have picked the model that misses the most late fires.

7.2 Precision and recall class by class



Figure 16. Precision and recall for each class and each model on 2023–2025.

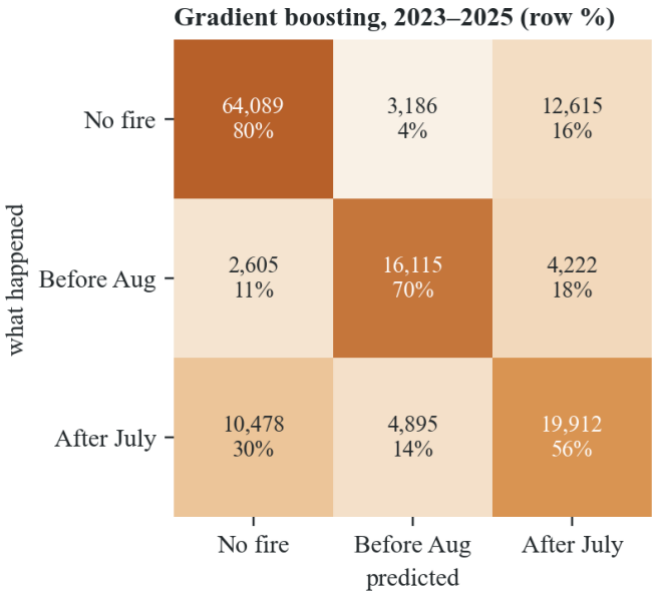


Figure 17. Confusion matrix for gradient boosting on 2023–2025, with counts and row percentages.

Figure 16 shows that every model finds late fire hardest. Gradient boosting reaches 0.830 precision and 0.802 recall on unburnt, 0.666 and 0.702 on early, and 0.542 and 0.564 on late. The confusion matrix in Figure 17 shows where the late errors go. Of 35,285 squares that burnt after July, the model caught 19,912 (56%). It called 10,478 of them no fire and 4,895 early. It also raised 12,615 false late alarms on squares that stayed unburnt.

The models differ in which mistake they prefer. The networks have the highest late recall (0.706 for the ANN, 0.688 for the LSTM) and the lowest late precision (0.459 and 0.474), so they catch more late fires at the cost of more false alarms. The random forest is the reverse, with the best late precision (0.599) and the worst late recall (0.384). Gradient boosting sits between them, with precision and recall both above 0.54, which is why it has the best F1.

Which model a fire manager should prefer depends on what a missed fire costs compared with a wasted burn. We could not settle that from the data. It is a decision for the people who carry the cost.

7.3 What class weighting did

Weighting changed the late class far more than the overall scores. For gradient boosting, late recall rose from 0.321 to 0.564 and late F1 from 0.420 to 0.553, while accuracy barely moved (0.724 to 0.725). For the random forest, late recall rose from 0.174 to 0.384. Without weights, the forest was close to giving up on late fire entirely. One model behaved differently. The unweighted LSTM had a slightly higher accuracy (0.738) than the weighted one (0.692), and almost the same F1, because weighting traded unburnt recall for late recall in nearly equal measure. We kept the weighted versions throughout, because missing late fire is the costlier error.

7.4 Overfitting

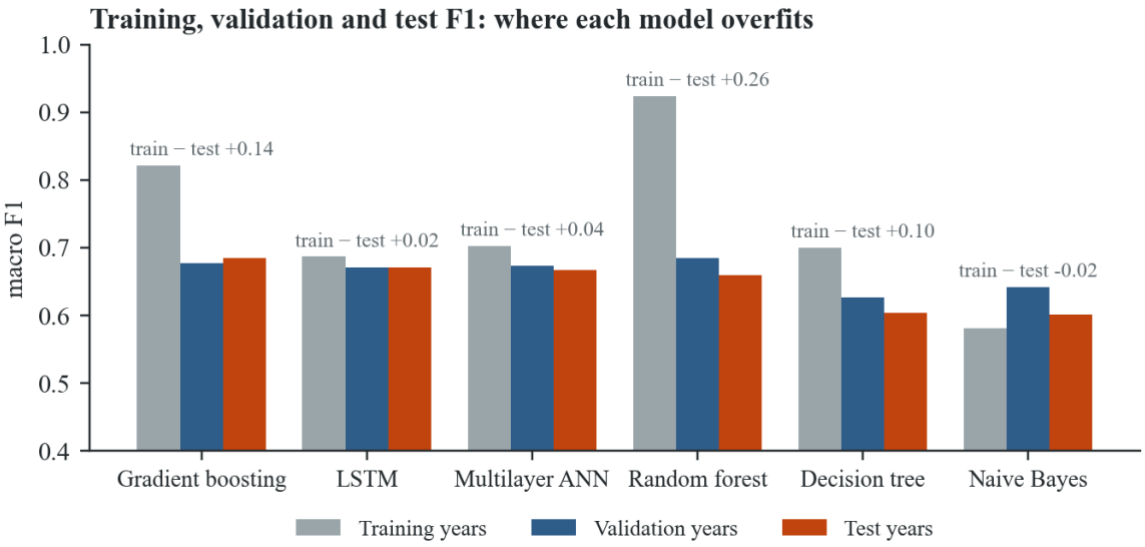


Figure 18. Macro F1 on the training, validation and test years for each model.

Figure 18 shows clear differences. The random forest overfits most, with 0.924 on its training years against 0.660 on the test years, a gap of 0.26. Its trees grow until a leaf holds as few as five rows, so they learn much of the training years by heart. Gradient boosting has a smaller gap (0.821 against 0.684), and the decision tree's gap is 0.10. The networks have the smallest gaps, 0.02 for the LSTM and 0.04 for the ANN, because early stopping halted them at epochs 3 and 11. Naive Bayes scores lower on its training years than on its test years, which is what a model too simple to overfit looks like. For every model, the validation and test scores are within 0.04 of each other, so the results held on a second set of unseen years.

7.5 Testing it sixty-six times

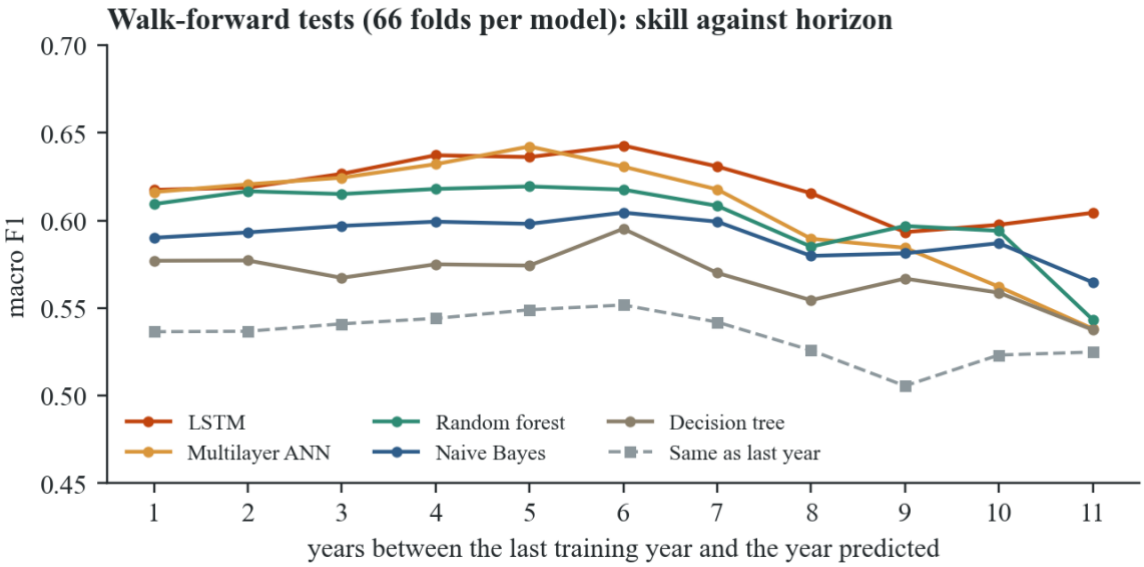


Figure 19. Walk-forward validation. Each line is the mean macro F1 of one model across 66 folds, by how far ahead it was asked to predict.

One good test period could be luck, so we repeated the test 66 times. A cut-off was set at each year from 2014 to 2024. A model was trained only on the years before the cut-off and then scored on every year after it. This walk-forward run was made before greenness was added, so it used the 36 fire-history and weather features, and it covered the five learners that existed at the time. Gradient boosting was added to the project later and is not in it. Across the 66 folds the LSTM averaged 0.625 (standard deviation 0.043), the ANN 0.618 and the random forest 0.611. The same-as-last-year baseline averaged 0.539. Naive Bayes beat the baseline in all 66 folds, the random forest in 65, and the LSTM and ANN in 64. The decision tree managed only 50, which matches its weaker test score.

The scores barely fade with distance. The LSTM scores 0.617 predicting one year ahead and 0.604 predicting eleven years ahead. Our explanation is that the strongest clue is the wet season just gone, which is known by 30 April however old the training data is. The model learns how grass and rain turn into fire, and that relationship does not change much from one decade to the next.

7.6 What the model relies on

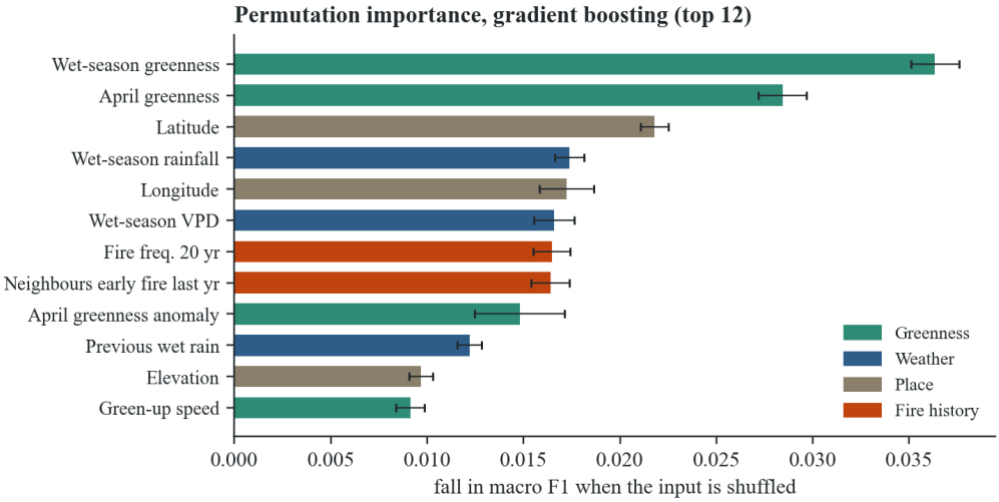


Figure 20. Permutation importance for gradient boosting: the fall in macro F1 when each input is shuffled, averaged over five repeats on 40,000 held-out squares.

We measured importance by permutation. Each input was shuffled in turn, which breaks its link with the label, and we recorded how far macro F1 fell (Breiman, 2001). This avoids the bias of the forest's built-in impurity importance, which favours features with many distinct values (Strobl et al., 2007). For gradient boosting the four strongest inputs are mean wet-season greenness (a fall of 0.036), April greenness (0.028), latitude (0.022) and wet-season rainfall (0.017). Eight of the top ten are greenness, weather or place features.

Nobody told the model that a wet season grows the grass that carries an October fire, and it found that relationship anyway. It ranks the same drivers that fire ecologists already rank first, which suggests it learned the problem and not quirks of the dataset. The low ranking of most fire-history features fits Figure 7. They are strongly correlated with each other, so when one is shuffled the model can still read the others.

7.7 The 2026 forecast

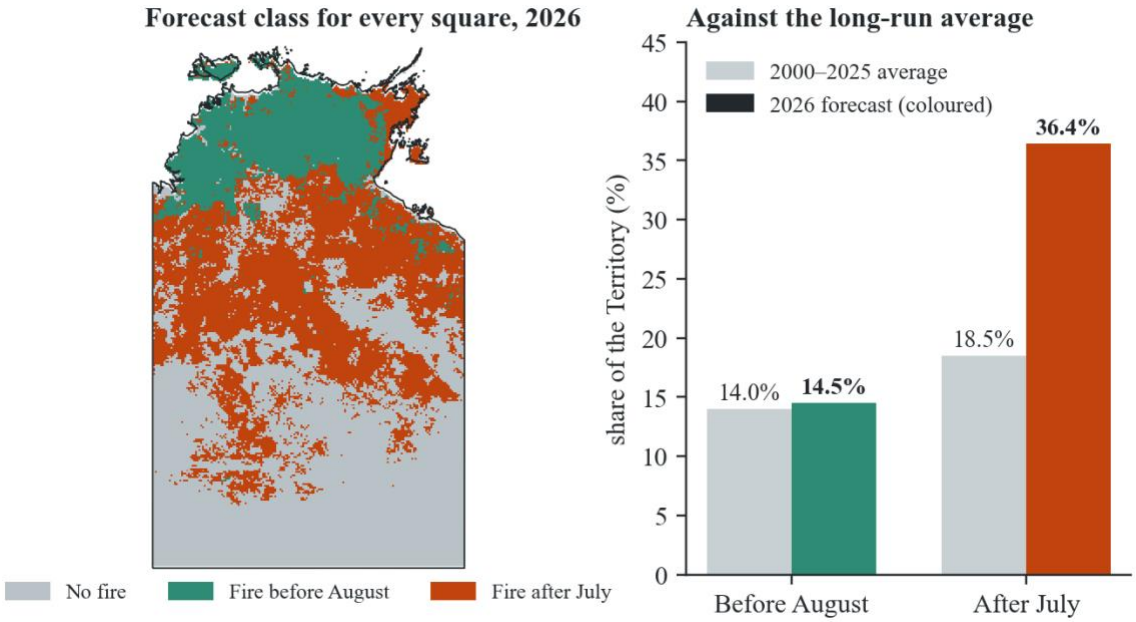


Figure 21. The 2026 forecast for every square, and the forecast shares against the 2000–2025 average.

Trained on every labelled year up to 2025, gradient boosting expects about 51% of the Territory to burn in 2026, with 14.5% burning before August and 36.4% after July. The 26-year average is 32.5%. Nearly all of the increase is late fire, which is about twice its long-run share of 18.5%. The model's mean confidence across squares is 0.733. It reads this from the inputs it ranks highest. The 2025–26 wet season brought 783 mm on average, 45% above the 2005–2019 mean and the second wettest in 22 years, and April 2026 was the greenest April in the record. More rain grew more grass, and more grass is more fuel. The map in Figure 21 puts most of the expected late fire in the interior savanna band, the same cluster that Figure 8 picked out as burning late. NAFI's 2026 mapping will show in 2027 whether the forecast was right.

7.8 Our overall understanding

The main lesson of the project is that the period you test on has a large effect on the result. The validation years were quiet (8.4% late fire), and the test years were heavy (25.5%), so the models were tuned on one kind of season and judged on another. That they held up, with validation and test scores within 0.04, says something about the models. It also means that the 2023–2025 numbers belong to a run of wet years, and a run of dry years would likely give different per-class scores. The walk-forward folds, which span 2015 to 2025, are the better guide to the long-run skill: around 0.62 for the best learners, against 0.54 for the baseline.

The second lesson is that the measure decides the winner. Ranked by accuracy, the random forest would have won while missing 62% of late fires. Ranked by macro F1, gradient boosting won with balanced precision and recall on every class. Precision and recall also turned out to be a choice for the people who use the forecast, more than a technical detail. A ranger group with crews to spare might prefer the networks' higher late recall. One that cannot afford wasted burns might prefer the forest's precision.

The third lesson is about where the remaining error comes from. Late fire is still the weakest class at 0.553 F1, and we think we know why. Whether a square burns late depends on events after 30 April, such as a lightning storm, a dropped cigarette or a September wind change. The 30 April rule keeps all of that out of the model on purpose. Some of the error is the price of making a forecast before the season starts, and no model trained under that rule can remove it.

The Week 7 lecture listed five pillars of ethical AI, and we used them as a check on the project. Fairness is less about people here, but the model is weakest in the arid south, where fires are rare, and it should not be trusted there as much as in the north. Robustness was tested with the walk-forward folds and the two leakage tests. Transparency and explainability are why the data, code and every chart are public, and why the site explains each prediction with the inputs behind it. The data contains no personal information. The model is meant to support the people who plan burning, and it should not replace them. Their knowledge of country is something the model has no feature for.

7.9 Limitations

Four limitations remain. Every score is measured against NAFI's own fire mapping, so any errors in that mapping are learned as if they were true. NAFI assigns each burnt pixel to the month that covers most of the interval in which it was detected, so labels close to the July and August boundary are approximate. All the tuning and the 23 feature experiments were judged on the same three validation years, so some of the gain may fit those years rather than fire in general. Finally, the model is fitted to the Northern Territory at 0.05 degrees. Whether it would work in the Kimberley or on Cape York is untested.

References

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281–305.

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.

Chollet, F., & others. (2015). *Keras* [Computer software]. https://keras.io

Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-1*(2), 224–227. https://doi.org/10.1109/TPAMI.1979.4766909

Didan, K. (2021). *MODIS/Terra vegetation indices 16-day L3 global 250 m SIN grid V061* [Data set]. NASA EOSDIS Land Processes Distributed Active Archive Center. https://doi.org/10.5067/MODIS/MOD13Q1.061

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment, 202*, 18–27. https://doi.org/10.1016/j.rse.2017.06.031

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Jeffrey, S. J., Carter, J. O., Moodie, K. B., & Beswick, A. R. (2001). Using spatial interpolation to construct a comprehensive archive of Australian climate data. *Environmental Modelling & Software, 16*(4), 309–330. https://doi.org/10.1016/S1364-8152(01)00008-1

Karim, A. (2026). *PRT565 Machine learning, artificial intelligence and algorithms: Lecture slides and tutorial notebooks, Weeks 1–11* [Unpublished course materials]. Charles Darwin University.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146–3154). Curran Associates.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory, 28*(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

North Australian Fire Information. (2026). *Northern Territory fire scar mapping* [Data set]. Darwin Centre for Bushfire Research, Charles Darwin University. https://firenorth.org.au

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Queensland Government. (2026). *SILO climate data: Data Drill* [Data set]. Long Paddock. https://www.longpaddock.qld.gov.au/silo/

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*(1), 81–106. https://doi.org/10.1007/BF00116251

Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig, F., & Dormann, C. F. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography, 40*(8), 913–929. https://doi.org/10.1111/ecog.02881

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*(6088), 533–536. https://doi.org/10.1038/323533a0

Russell-Smith, J., Cook, G. D., Cooke, P. M., Edwards, A. C., Lendrum, M., Meyer, C. P., & Whitehead, P. J. (2013). Managing fire regimes in north Australian savannas:

Applying Aboriginal approaches to contemporary global problems. *Frontiers in Ecology and the Environment, 11*(s1), e55–e63. https://doi.org/10.1890/120251

Russell-Smith, J., Yates, C. P., Whitehead, P. J., Smith, R., Craig, R., Allan, G. E., Thackway, R., Frakes, I., Cridland, S., Meyer, M. C. P., & Gill, A. M. (2007). Bushfires 'down under': Patterns and implications of contemporary Australian landscape burning. *International Journal of Wildland Fire, 16*(4), 361–377. https://doi.org/10.1071/WF07018

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), Article e0118432. https://doi.org/10.1371/journal.pone.0118432

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*, 1929–1958.

Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics, 8*, Article 25. https://doi.org/10.1186/1471-2105-8-25